CCE unit=3
Linked list

Page No. :
Date :

Title :- Autononous Warehouse Robot Task management system.

mini project using stack and queue in c++

project Type:. software based simulation.

* Introduction

An Autonomous warehouse Robot task mono. gement system is a software project That simulates how robot perform task in a modern warehouse. warehouse robot may collect products. Transport goods. and deliver packages to specified location when several tasks. arrive They must be organized and processed systematically. ih This project, a queue stores pending task in first in. first out (FIFo) order. while a stack stores completed task history so The most recently completed task can be viewed first. The menu driven c++ program allows user to add tasks. process tasks. view pending works. and inspect or remove the latest history entry.

problem statement

A warehouse may receive many robot tasks at The sam time. such as picking products. moving boxes and delive

packages. without and organized system. pending task can become difficult To track a software system is Therefore. requird To maintain a queue of pending tasks. process task in arrival order and store Completed - task history using a stack.

* objectives
- manage warehouse robot tasks at the same time.
- use a queue to maintain completed task history in FIFO order.
- Display pending tasks and completed tasks.
- Handle empty queue and stack condition safely.
- Demonstrate practical applications of stack and queue in automation.

Data structure used

queue (FIFO) :. The first task inserted is The first task processed. main operations. are enqueue (add) deque. (process / remove) and display.

stack (LIFO):. The latest Completed task is at The top. main operation are push (store) pop (remove The latest history entry). and display.

Page No. :
Date :

Algorithm

1. Start
2. initialize an empty queue named pending Tasks and an empty stack named completed tasks.
3. Display The main menu and accept The user's choice.
4. if choice = 1. read a task description and enque if it is not empty.
5. if choice = 2: Check The queue. If empty show a message: otherwise dequeue the oldest task. simulate processing. and push it onto the completed-task stack.
6. if choice = 3: display all pending tasks by copying The stack and popping items for The copy.
7. if choice = 4: display completed-task history by copying The stack and popping items from The copy.
8. if choice = 5: Check The stack. If empty show a message. otherwise display and pop The latest history entry.

Page No. :
Date :

(9) if choice = 6: display The exit message and terminate.

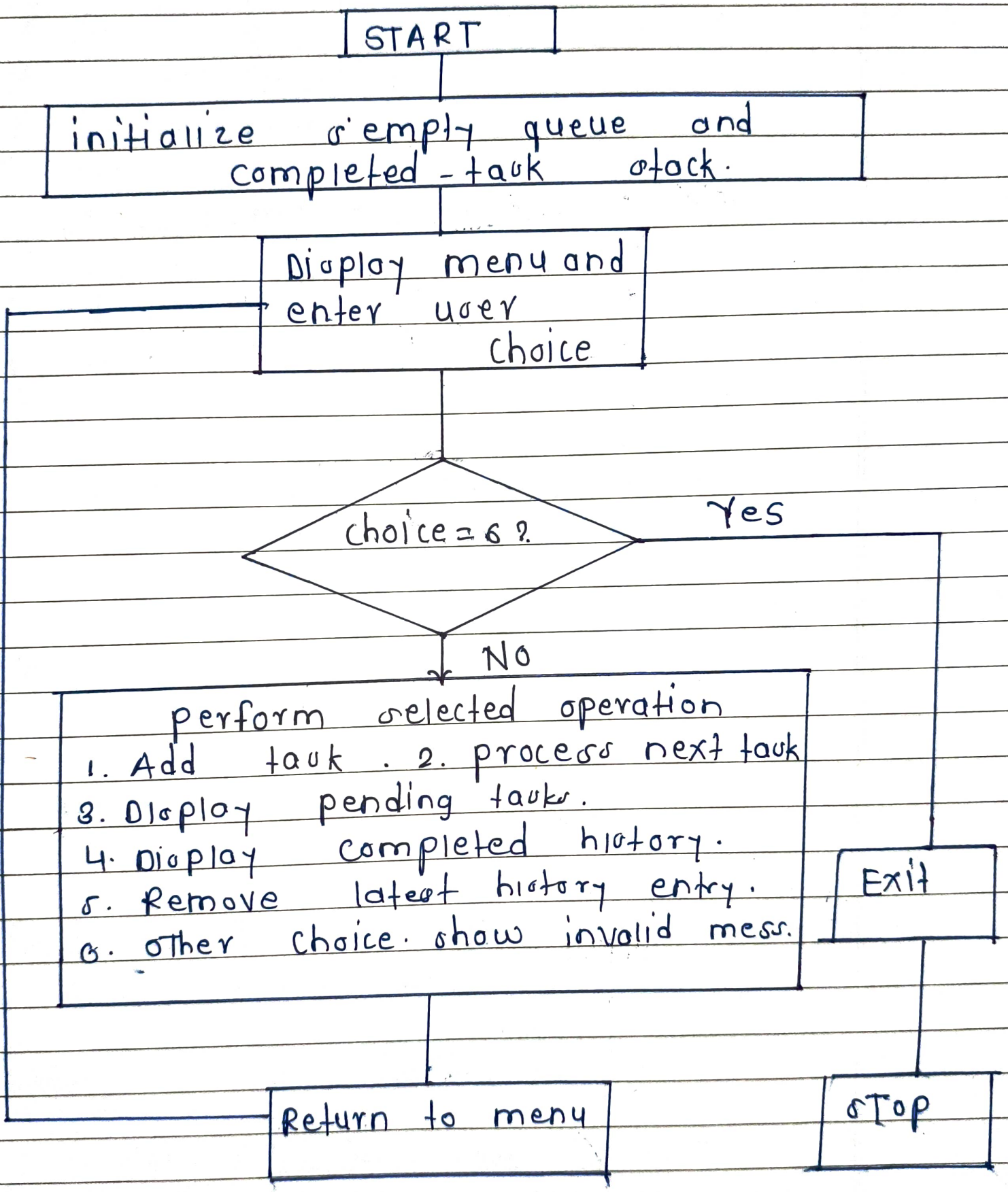
(10) For any other choice, display an invalid choice message..

(11) Repeat steps 3-10 until The user select exit

(12) Stop.

Flowchart

Page No. :
Date :



Page No. :
Date :

Program

```
#include <iostream>
#include <queue>
#include <stack>
#include <string>
#include <limits>
using namespace std;

int main ()
{
    queue <string> pending tasks;
    stack <string> CompletedTasks;
    int choice;
    string task;

do
{
  cout <<
"\n ==========\n";

cout <<" AUTONOMOUS WAREHOUSE ROBOT
        SYSTEM \n";

cout << " ==========\n";

    cout << "1. Add New Task\n";
    cout <<" 2. process Next task\n";
    cout <<" 3. display pending task\n";
    cout <<" 4. Display completed task\n";
```

```
cout <<"5. Remove latest history
        Entry \n";
cout <<" 6. Exit\n";
cout <<" Enter your choice :";

if (!(cin >> choice))
{

cout <<" Invalid input. Enter a number
\n":
    cin. clear ();

cin. ignore (numeric_limits < streamsize>:
 max(), '\n');
          Continue;
    }

cin. ignore (numeric_limits < streamsize >::
 max (), '\n');

    switch (choice)
   {
   case 1:
      cout<<" Enter task description:";
      getline (cin, task);

      if ( task . empty) () )
       cout << "Task description cannot Empty\n"
```

Page No. :
Date :

```
Case 2:
  if (! pendingTasks.empty())
{
    task = pendingTasks.front();
    pendingTask.pop();

CompletedTasks.push(task);

    cout << "Robot is processing:" << task << endl;
    cout << "Task completed successfully\n";
  }
  else
{
    cout << "No pending task!\n";
}
  break;

Case 3:
 if ( pendingTasks.empty())
 {
   cout << "No pending tasks!\n";
 }
 else
 {
    queue<string> temp = pendingTasks;
    cout << "pending Tasks:\n";
```

```
        while (!temp.empty())
        {
            cout << temp.front() << endl;
            temp.pop();
        }
    }

    break;

case 4:
    if (completedTasks.empty())
    {
        cout << "No completed task:\n";
        while (!temp.empty())
        {
            cout << temp.ptop() << endl;
            temp.pop();
        }
    }
    break;
case 5:
    cout << "system closed succesfully\n";
    break;
default: cout << "Invalid choice!\n";
}
} while (choice != 5);
return 0;
}
```